Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#5)

Use of Venus Simulator for Register Level Execution of RISC-V Control Instructions

Student Name: **AHMED JAVED QURESHI** Roll No:**24i-6554** Section: **CE-5A**

Date performed: **15 Sep, 2026**

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____
Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

Task 1:

Description:

The task was to go through Table 1 and Table 2 and say which instructions are actual and which are pseudo. I checked each one against the RV32I base set, which only has beq, bne, blt, bge, bltu and bgeu for branches. So beq, bne, blt and bge are actual, while ble and bgt are pseudo because the assembler just swaps the registers and uses bge or blt. j is also a pseudo, it becomes jal x0, and jal and jalr are actual jumps. To confirm it I assembled code in Venus and compared the original code column with the basic code column.

Code:

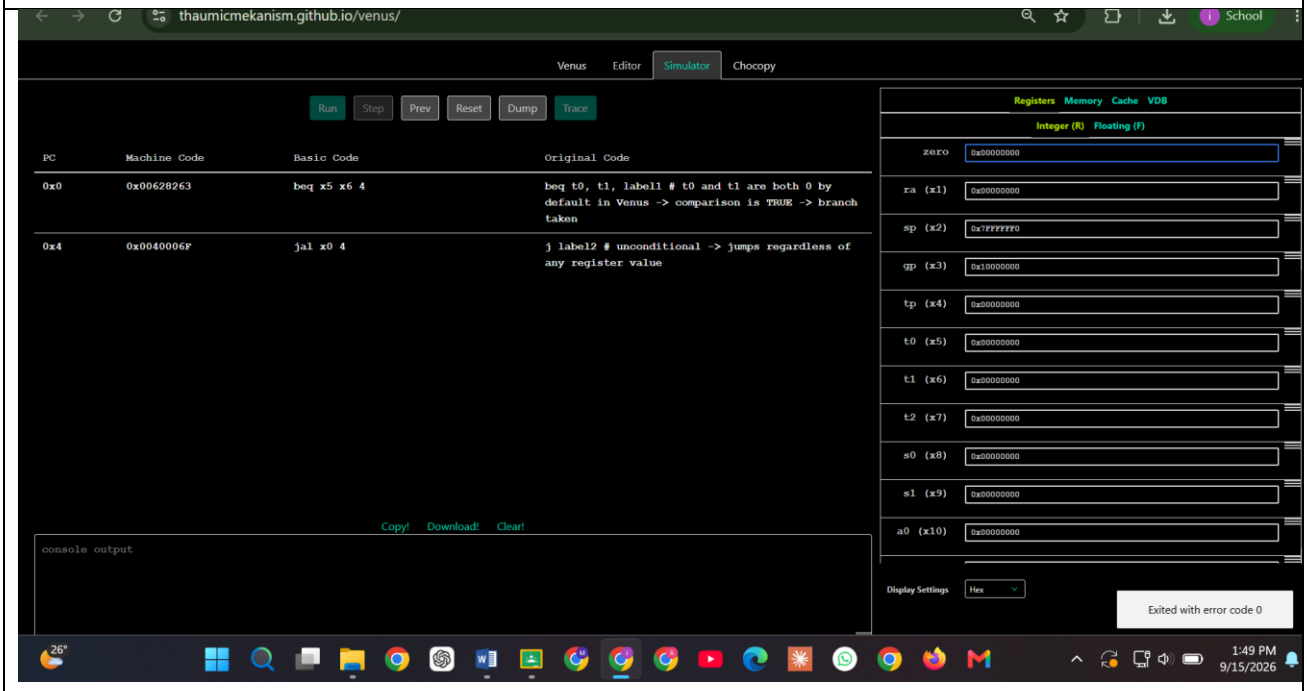
Instruction	Actual or Pseudo	Why
beq	Actual	real B-type opcode in RV32I
blt	Actual	real B-type opcode in RV32I
ble	Pseudo	no dedicated opcode — assembler emits bge rs2,rs1 (operands swapped)
bgt	Pseudo	assembler emits blt rs2, rs1 (operands swapped)
bge	Actual	real B-type opcode
bne	Actual	real B-type opcode
J	Pseudo	expands to jal x0, target

Output:

RV32I only defines a fixed set of real opcodes (beq, bne, blt, bge, bltu, bgeu, jal, jalr). Anything outside that list is a pseudo-instruction the assembler expands into a real one. In Venus you can confirm this directly — assemble the code and check the generated machine code/disassembly listing; a pseudo-instruction shows up rewritten as the real instruction it maps to.

Task 2:

Description:
For this task I wrote a small snippet with only beq and j to see how a conditional branch is different from a jump. I didn't set t0 and t1, so both are 0 by default in Venus and beq t0, t1, label1 finds them equal and takes the branch. At label1 I put j label2, which jumps every time without checking anything. After running it I saw both instructions in the basic code column, beq as it is and j as jal x0. So beq depends on a condition and j does not.
Code:
beq t0, t1, label1 label1: j label2 label2:
Output:



Task 3:

Description:
Here I ran Code 1 and Code 2 in Venus and wrote a comment on every line. In Code 1, s0 becomes 4 and s1 becomes 6, so beq s0, s1, Fin is false and the branch is not taken. It goes to the True part, adds them to get s2 = 10, and then j end skips the Fin block. In Code 2 both s0 and s1 are 4, so the branch is taken and it lands on Fin. The add and j are skipped and sub gives s2 = 0.
Code:

CODE 1:

- addi s0, s0, 4 → s0 starts at 0 (Venus default), adds 4 → s0 = 4
- addi s1, s1, 6 → s1 starts at 0, adds 6 → s1 = 6
- beq s0, s1, Fin → compares s0 (4) and s1 (6). They are not equal, so this branch is **not taken**; execution falls through to the next line
- True: → this is just a label marking a location in the code, it does nothing by itself
- add s2, s1, s0 → since beq wasn't taken, this line runs: s2 = 6 + 4 = 10
- j end → unconditionally jumps to the end label, skipping the Fin block below
- Fin: → label marking the "not equal" landing point — not reached in this run because of the j end above

sub s2, s1, s0 → not executed, since j end skipped over it

- end: → label marking the end of the program, execution stops here

CODE 2:

1. addi s0, s0, 4 → s0 starts at 0, adds 4 → s0 = 4
2. addi s1, s1, 4 → s1 starts at 0, adds 4 → s1 = 4
3. beq s0, s1, Fin → compares s0 (4) and s1 (4). They are equal, so this branch **is taken**; execution jumps straight to Fin, skipping everything in between
4. True: → label, never reached this run because the branch above jumped past it
5. add s2, s1, s0 → skipped entirely, never executed
6. j end → also skipped, never executed
7. Fin: → this is where execution lands after the taken branch
8. sub s2, s1, s0 → runs: s2 = 4 - 4 = 0

9. End: → label marking end of program, execution stops here

Output:

Task 4:

Description:

In this task I had to convert the given C if else code into RISC-V assembly with a = 5, b = 10, d = 4 and e = 3. I loaded the four values into registers using addi first. Then I used beq to compare a and b, and j to go to the else part when they are not equal. Since 5 is not equal to 10, the else part ran and did c = d + e, stored in t4. After running it in Venus, t4 showed 0x7.

Code:

```
addi t0, zero, 5     # a = 5

addi t1, zero, 10     # b = 10

addi t2, zero, 4      # d = 4

addi t3, zero, 3       # e = 3



beq t0, t1, IfTrue       # if (a == b) go to IfTrue

j Else                  # else go to Else



IfTrue:

sub t4, t2, t3              # c = d - e

j End



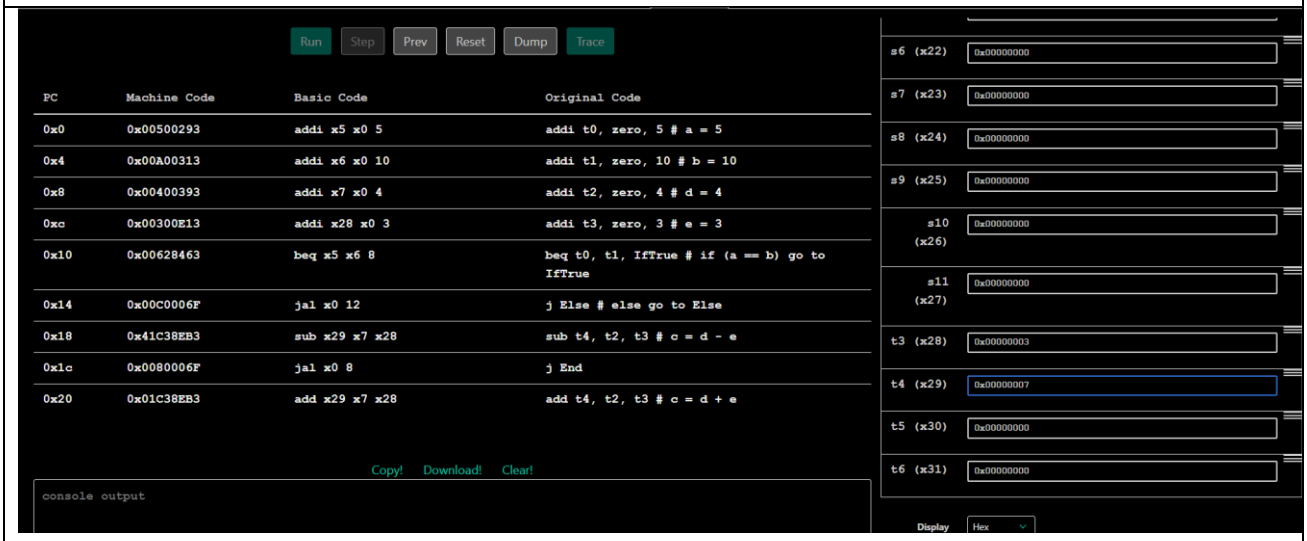
Else:

add t4, t2, t3               # c = d + e



End:
```

Output:



Task 5:

Description:

The task was to shift a number to the left two times. I put 2 in t0 using addi and then used slli t0, t0, 1, which shifts it left by one bit, so t0 became 4. I ran slli again and t0 became 8. Each shift works like multiplying by 2. I checked the value of t0 in the register panel after each instruction while stepping through in Venus.

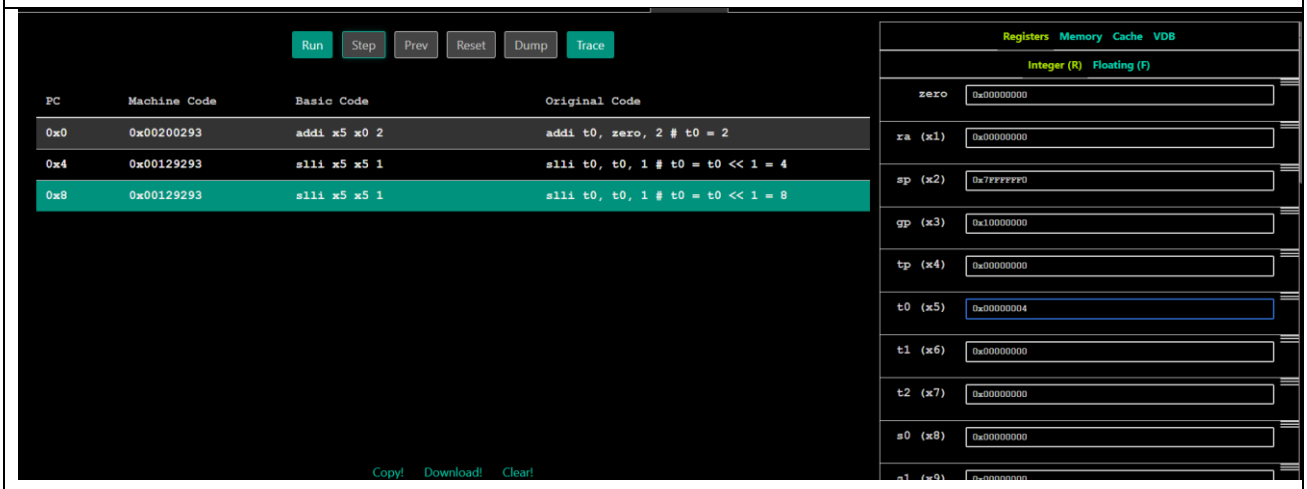
Code:

```
addi t0, zero, 2     # t0 = 2

slli t0, t0, 1       # t0 = t0 << 1 = 4

slli t0, t0, 1       # t0 = t0 << 1 = 8
```

Output:



Task 6:

Description:

Here I converted the for loop (i from 1 to 5, increment counter) into assembly. I used s0 for the counter and set it to 0, s1 for i starting at 1, and t0 to hold the limit 5. At the start of the loop, bgt

checks if i is greater than 5 and jumps to end if so. Otherwise I did counter++ and i++ and used j loop to go back up. The loop ran 5 times and finished with s0 = 5 and s1 = 6.

Code:

```
addi s0, zero, 0     # counter = 0

addi s1, zero, 1      # i = 1

addi t0, zero, 5       # loop limit = 5



loop:

bgt s1, t0, end         # if (i > 5) exit loop

addi s0, s0, 1           # counter++

addi s1, s1, 1            # i++

j loop



end:
```

Output:



Task 7:

Description:
This task was to write the while loop that fills num[i] with i + 5 while i is less than 5. I declared the array num with 10 words in the data section and loaded its address into t0 using la. Then I used t1 as i and t2 as 5, and bge to leave the loop when i reaches 5. In each round I calculated i + 5, shifted i by 2 to get the byte offset, added it to the base address and stored the value with sw. So the first five elements became 5, 6, 7, 8 and 9.
Code:
.data num: .word 0, 0, 0, 0, 0, 0, 0, 0, 0, 0

```
.text

la t0, num

addi t1, zero, 0

addi t2, zero, 5



loop:

bge t1, t2, end

addi t3, t1, 5

slli t4, t1, 2

add t5, t0, t4

sw t3, 0(t5)

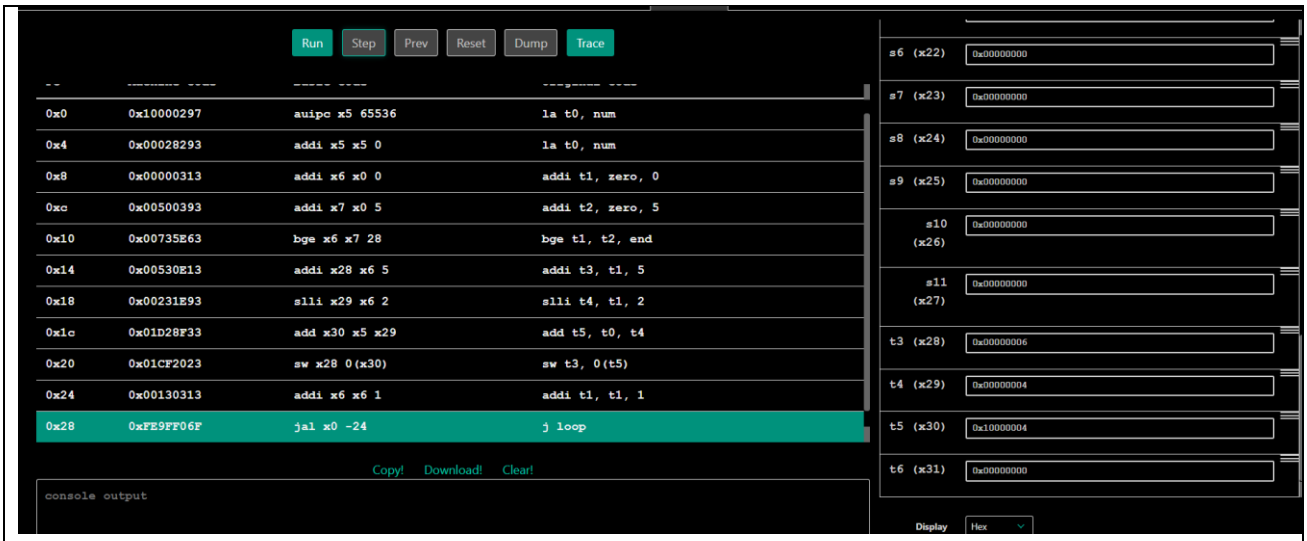
addi t1, t1, 1

j loop



end:
```

Output:



Task 8:

Description:
For this task I wrote the nested for loops in assembly, with x from 0 to 2 on the outside and y from 0 to 1 on the inside. I used s1 for x and s2 for y, and set s2 back to 0 every time the outer loop starts. The inner loop does add s0, s1, s2 and then increments y, and when it finishes I increment x and jump back to outer. Counter = x + y overwrites the old value each time, so s0 ends with the last one, 2 + 1 = 3.

Code:

```
addi s0, zero, 0

addi s1, zero, 0

addi t0, zero, 3



outer:

bge s1, t0, done

addi s2, zero, 0

addi t1, zero, 2



inner:

bge s2, t1, nextx

add s0, s1, s2

addi s2, s2, 1

j inner



nextx:

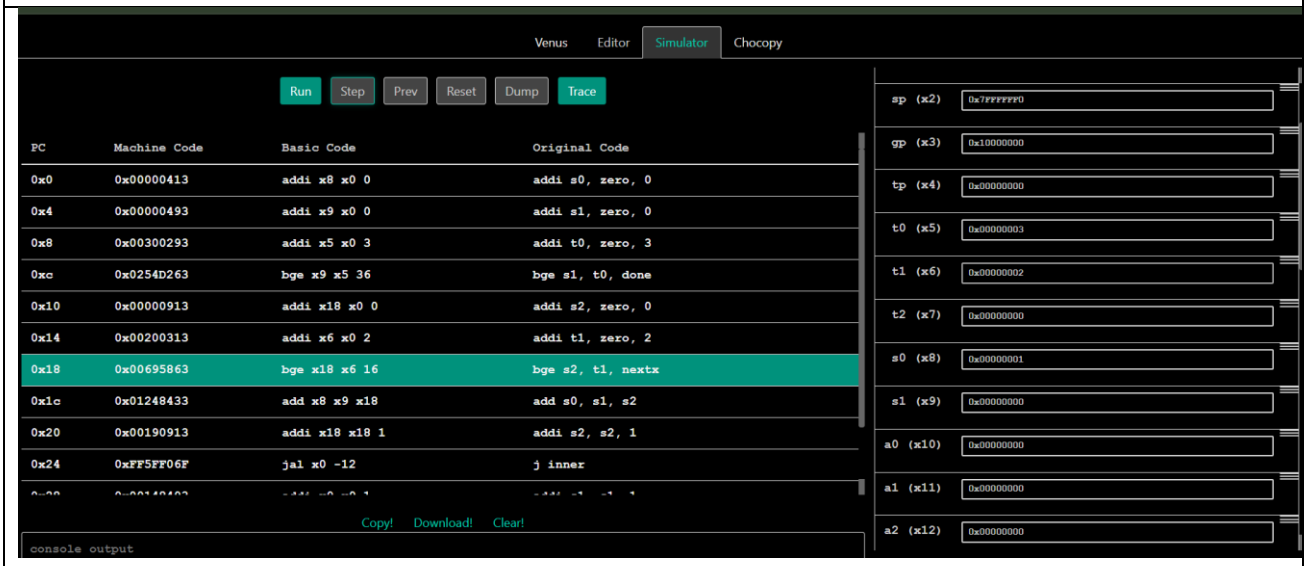
addi s1, s1, 1

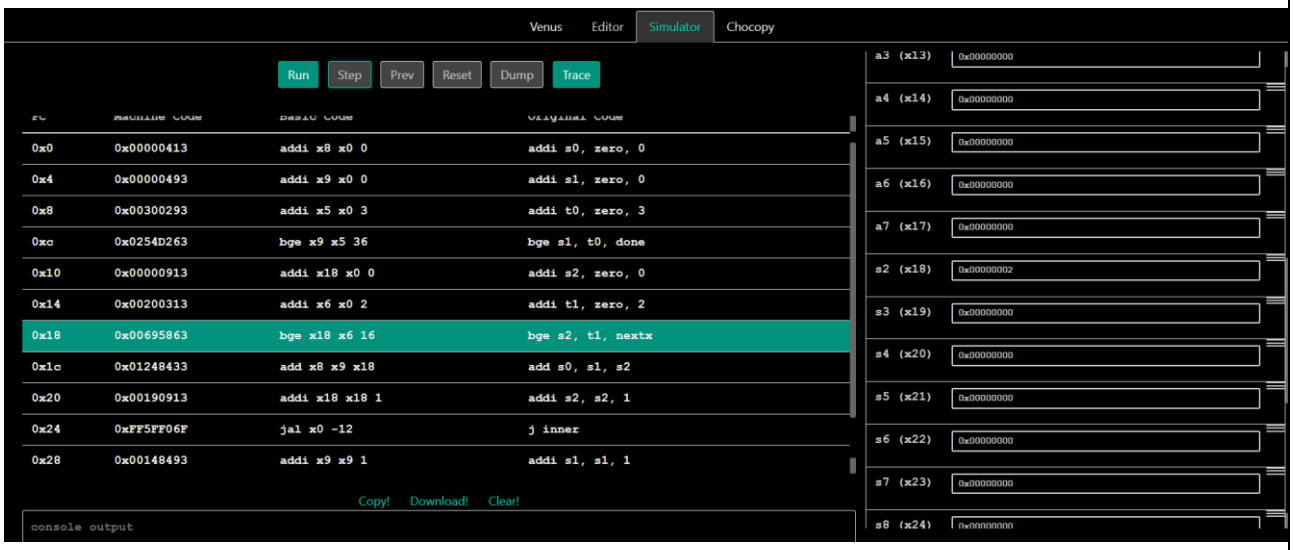
j outer



done:
```

Output:


Venus
Editor
Simulator
Chocopy
Run
Step
Prev
Reset
Dump
Trace
PC
Machine Code
Basic Code
Original Code
0x0 0x00000413 addi x8 x0 0 addi s0, zero, 0
0x4 0x00000493 addi x9 x0 0 addi s1, zero, 0
0x8 0x00300293 addi x5 x0 3 addi t0, zero, 3
0xc 0x0254D263 bge x9 x5 36 bge s1, t0, done
0x10 0x00000913 addi x18 x0 0 addi s2, zero, 0
0x14 0x00200313 addi x6 x0 2 addi t1, zero, 2
0x18 0x00695863 bge x18 x6 16 bge s2, t1, nextx
0x1c 0x01248433 add x8 x9 x18 add s0, s1, s2
0x20 0x00190913 addi x18 x18 1 addi s2, s2, 1
0x24 0xFF5FF06F jal x0 -12 j inner
Copy!
Download!
Clear!
console output
sp (x2) 0x7FFFFFF0
gp (x3) 0x10000000
tp (x4) 0x00000000
t0 (x5) 0x00000003
t1 (x6) 0x00000002
t2 (x7) 0x00000000
s0 (x8) 0x00000001
s1 (x9) 0x00000000
a0 (x10) 0x00000000
a1 (x11) 0x00000000
a2 (x12) 0x00000000


Venus
Editor
Simulator
Chocopy
Run
Step
Prev
Reset
Dump
Trace
0x0 0x00000413 addi x8 x0 0 addi s0, zero, 0
0x4 0x00000493 addi x9 x0 0 addi s1, zero, 0
0x8 0x00300293 addi x5 x0 3 addi t0, zero, 3
0xc 0x0254D263 bge x9 x5 36 bge s1, t0, done
0x10 0x00000913 addi x18 x0 0 addi s2, zero, 0
0x14 0x00200313 addi x6 x0 2 addi t1, zero, 2
0x18 0x00695863 bge x18 x6 16 bge s2, t1, nextx
0x1c 0x01248433 add x8 x9 x18 add s0, s1, s2
0x20 0x00190913 addi x18 x18 1 addi s2, s2, 1
0x24 0xFF5FF06F jal x0 -12 j inner
0x28 0x00148493 addi x9 x9 1 addi s1, s1, 1
Copy!
Download!
Clear!
console output
a3 (x13) 0x00000000
a4 (x14) 0x00000000
a5 (x15) 0x00000000
a6 (x16) 0x00000000
a7 (x17) 0x00000000
s2 (x18) 0x00000002
s3 (x19) 0x00000000
s4 (x20) 0x00000000
s5 (x21) 0x00000000
s6 (x22) 0x00000000
s7 (x23) 0x00000000